ECPrefix: Erasure-Coded Prefix KV Cache for Optimizing Load Balancing in Large Language Model Serving

Feifan Liu
ffliu@hust.edu.cn
Huazhong University of Science and Technology
Wuhan, China

Yuchong Hu*
yuchonghu@hust.edu.cn
Huazhong University of Science and Technology
Wuhan, China

Mingqi Li
mingqili1006@hust.edu.cn
Huazhong University of Science and Technology
Wuhan, China

Lin Wang
wanglin2021@hust.edu.cn
Huazhong University of Science and Technology
Wuhan, China

Chenxuan Yao
deadfffool@hust.edu.cn
Huazhong University of Science and Technology
Wuhan, China

Zhenghao Yu
d202581835@hust.edu.cn
Huazhong University of Science and Technology
Wuhan, China

Dan Feng
dfeng@hust.edu.cn
Huazhong University of Science and Technology
Wuhan, China

Abstract

Prefix KV cache is widely used to accelerate LLM serving by trading more storage for less computation, and state-of-the-art methods often replicate hotspot caches for load balancing. However, we observe that the few nodes that have cache replicas still lead to severe load imbalance. This paper presents ECPrefix : a new prefix KV cache framework based on erasure coding (instead of replication), which distributes encoded blocks of hot prefix caches (organized as profile-guided objects) across nodes, along with adaptive striping and pipelined reading optimizations. Evaluation shows that ECPrefix reduces TTFT by up to 52.3% over existing systems.

Keywords

Load Balance, Erasure Coding, LLM Serving.

ACM Reference Format:
Feifan Liu, Yuchong Hu, Mingqi Li, Lin Wang, Chenxuan Yao, Zhenghao Yu, and Dan Feng. 2026. ECPrefix: Erasure-Coded Prefix KV Cache for Optimizing Load Balancing in Large Language Model Serving. In *63rd ACM/IEEE Design Automation Conference (DAC '26), July 26–29, 2026, Long Beach, CA, USA.* ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/3770743.3804204

1 Introduction

With the rapid development of large language models (LLMs), Generative AI such as ChatGPT [14] and DeepSeek [11] have achieved excellent results in areas like chatbots [9], document summarization [28], and code generation [6]. To enhance performance, *prefix KV cache*, which caches the attention Key-Value tensors corresponding to shared input prefixes across different requests, is widely used by trading storage for computation [3, 5, 7, 17]. State-of-the-art prefix KV cache systems (e.g., [17]) often replicate hotspot caches to avoid straggler nodes for load balancing as only 10% of KV cache contributes to 77% of reuse requests [24]. However, we observe that the nodes storing these hot KV cache replicas may still lead to severe load imbalance since replication's high redundancy incurs a too limited number of replicas to avoid stragglers (see §3).

*Corresponding Author.

DAC '26, Long Beach, CA, USA

ACM ISBN 979-8-4007-2254-7/2026/07
https://doi.org/10.1145/3770743.3804204

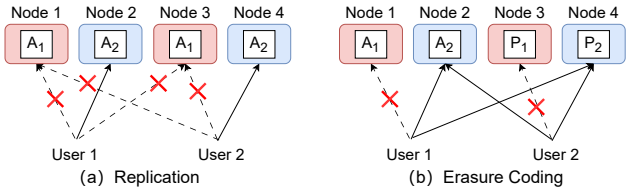


Figure 1: Erasure coding outperforms replication in terms of load balancing under stragglers (nodes in red refer to stragglers).

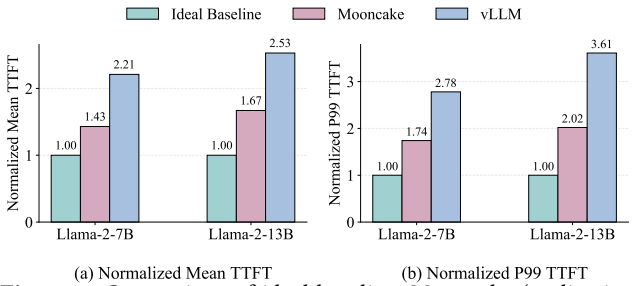
Erasure coding (EC) [8, 12, 18] provides a promising alternative to tolerate more stragglers than replication under the same overhead in clusters. Specifically, Figure 1(a) shows that when two original data blocks A_1 and A_2 are replicated across four nodes, where Node 1 and Node 3 become stragglers, users cannot obtain A_1 immediately. Alternatively, Figure 1(b) shows that A_1 and A_2 are erasure coded using a (2+2) Reed-Solomon code [25] with two parity blocks P_1 and P_2, also consuming four blocks same as in Figure 1(a), but users can fast decode A_1 from A_2 and P_2, showing the potential for better load balancing. Despite the promise of EC, how to design coding details for optimizing load balancing in LLM serving remains unexplored.

This paper presents ECPrefix (§4), a new erasure-coded prefix KV cache framework based on erasure coding (instead of replication), which organizes hot prefix caches into objects using a profile-based object size, and encodes objects into blocks distributed across nodes. Further, ECPrefix employs adaptive striping and pipelined reading optimizations to address the problem of selecting from multiple potential stripe sizes and the challenge of additional computational overhead. Evaluation (§5) shows ECPrefix reduces TTFT (time to first token) by up to 52.3% over existing systems, demonstrating significant latency reduction and enhanced load balancing.

2 Background and Related Work

2.1 Prefix KV cache and Load Imbalance

Prefix KV cache. Prefix KV cache is a specialized Key-Value (KV) cache for large language model (LLM) inference [7], storing precomputed Key-Value vector pairs of contextual prefix sequences from the prefill stage. It reuses these intermediate results in subsequent decoding to eliminate redundant computations, reduce memory bandwidth overhead, and enhance inference efficiency, particularly for multi-turn conversations, batch inference, and long-text reasoning with shared or fixed prefixes. Mooncake [17] achieves a widely



(a) Normalized Mean TTFT (b) Normalized P99 TTFT

Figure 2: Comparison of ideal baseline, Mooncake (replication) and vLLM (no replication by default) in latency. Note that all the results are normalized by comparing them to the ideal baseline.

used global cache mechanism across nodes with a higher cache hit rate by pooling the CPU, DRAM, and SSD of GPU clusters.

Load Imbalance in LLM Serving. The popularity of the global prefix KV cache varies drastically across nodes, which may incur load imbalance across nodes. Only 10% of the KV cache contributes to 77% of the *reuse* requests [24]. The bandwidth utilization rate of the nodes where these hotspot KV cache are stored is close to the physical upper limit [24]. Significant differences in bandwidth utilization among different nodes render load balancing a critical challenge for improving performance in LLM serving.

***r*-Selective Replication.** To address load imbalance, recent prefix KV cache systems (e.g., Mooncake [17], CloudMatrix384 [15]) often employ *r-selective replication*, which selects hot KV caches and stores r replicas for them into r nodes. We will analyze replication still fundamentally causes load imbalance in LLM serving in §3.1.

2.2 Erasure Coding Techniques

Basics. Erasure Coding (EC) is a highly effective data redundancy method designed to minimize storage overhead [8, 19]. An (n, k) EC begins by organizing data into fixed-size original *objects*, dividing each object into k *data blocks*, and encoding them into m *parity blocks*, collectively constituting a *stripe* comprising the $n = k + m$ blocks. Among the most widely-utilized instantiations of erasure coding are Reed-Solomon (RS) codes [25], which encode data blocks into parity blocks by Vandermonde matrix [1] using linear algebraic operations over finite fields (e.g., GF(2^8)).

Arbitrary reconstruction property. A fundamental property of EC is to decode any data blocks using any k of n blocks—whether data or parity—thereby providing tolerance for any $n - k = m$ straggler blocks, which we call *arbitrary reconstruction property*. This property will inspire the main idea of our ECPrefix (§4).

EC for load balancing. Based on the arbitrary reconstruction property, EC has significantly more choices than selective replication ($(C(n, k))$ vs. r) to avoid the straggler nodes. For example, EC-Cache [18], which also applies EC to traditional storage clusters for improving load balance. Therefore, we will analyze the balance benefits of EC over replication in §3.2 and aim to design EC for optimizing load balancing in §4.2.

3 Analysis

In this section, we first analyze the load imbalance of replication (§3.1), and then analyze the load balance benefits of erasure coding compared to replication (§3.2).

3.1 Analysis of Load Imbalance of Replication

State-of-the-art methods [17] address load imbalance via hot KV cache replication, yet this approach suffers from inherent limitations—load imbalance across nodes remains incompletely eliminated. The available memory capacity for prefix KV cache is constrained without impacting normal computational performance. Consequently, the high redundancy of replication limits the number of replicas for hotspot prefix KV cache, which only distributes the load pressure into too few nodes to effectively mitigate stragglers. Furthermore, the hotspot and long prefix cache in a very small number of nodes will constantly consume significant bandwidth during the prefill phase, which may turn these nodes into stragglers and result in a severe tail latency.

To verify the impact of replication on load imbalance, we take a measurement analysis as follows. We evaluate LLaMa-2 [22] on 8 nodes with the ShareGPT dataset [21]. Detailed setups are shown in §5.1. We let the case without load imbalance (i.e., unlimited bandwidth) be the ideal baseline. We compare Mooncake (with replication enabled) and vLLM (a popular LLM serving framework without replication supported by default) to examine how replication affects load imbalance in terms of inference performance (TTFT). Figure 2 shows that even if replication mitigates load imbalance (Mooncake vs. vLLM), its performance still falls far short of the ideal baseline with unlimited bandwidth (no load imbalance). For two models, compared to the ideal baseline, Mooncake slows the performance by 31% $\sim$ 40% in mean TTFT and 43% $\sim$ 50% in P99 TTFT. Additionally, performance loss becomes more pronounced as models grow larger.

3.2 Analysis of Load Balance Benefits of EC

Inspired by §2.2, we examine erasure coding to explore if it can help achieve better load balancing than replication in LLM serving. Its core principle directly addresses this challenge: by strategically distributing prefix KV cache into encoded blocks across nodes, erasure coding can avoid the stragglers more effectively.

To elaborate on how erasure coding achieve better load balance over selective replication, we take a theoretical analysis as follows:

Assumptions. Assuming there are N nodes and M prefix cache objects to be evenly distributed across nodes [17], where w_i represents the popularity of the i-th object and k represents the number of blocks in erasure coding.

Model. For replication in an arbitrary node, its total load L_{R} is the sum of load contributions from all objects; for erasure coding, each object is split into k blocks, so the total load L_{EC} of a node is the sum of contributions from all blocks, meaning that:

$$\begin{cases} L_{\mathrm{R}} = \sum_{i=1}^{M} w_i \cdot X_{i,\mathrm{R}} \\ L_{\mathrm{EC}} = \sum_{i=1}^{M} \sum_{j=1}^{k} \frac{w_i}{k} \cdot X_{i,j,\mathrm{EC}} \end{cases} \tag{1}$$

where $X_{i,\mathrm{R}}$ and $X_{i,j,\mathrm{EC}}$ are a Bernoulli random variables [17]. $X_{i,\mathrm{R}} = 1$ (probability $p = \frac{1}{N}$) if the i-th object is assigned to this node, and $X_{i,\mathrm{R}} = 0$ (probability $1 - p$) otherwise.

The variance of replication can be calculated:

$$\mathrm{Var}(L_{\mathrm{R}}) = \sum_{i=1}^{M} \mathrm{Var}(w_i \cdot X_{i,\mathrm{R}}) = \sum_{i=1}^{M} w_i^2 \cdot \frac{1}{N}\left(1 - \frac{1}{N}\right) \tag{2}$$

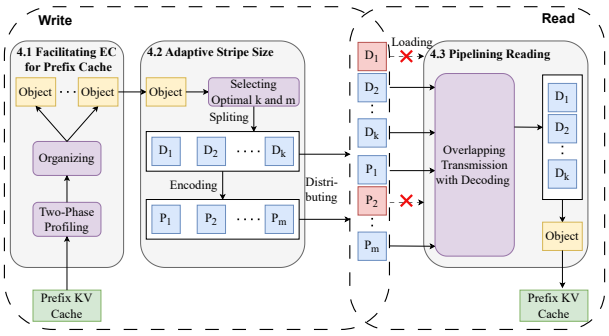


Figure 3: Architecture of ECPrefix .

The $X_{i,j,\mathrm{EC}}$ is the same variable as $X_{i,\mathrm{R}}$, so the following result can be obtained:

$$\mathrm{Var}(L_{\mathrm{EC}}) = \sum_{i=1}^{M}\sum_{j=1}^{k} \mathrm{Var}\left(\frac{w_i}{k}\cdot X_{i,j,\mathrm{EC}}\right) = \sum_{i=1}^{M} w_i^2 \cdot \frac{1}{k}\cdot\frac{1}{N}\left(1-\frac{1}{N}\right) \tag{3}$$

$$\frac{\mathrm{Var}(L_{\mathrm{EC}})}{\mathrm{Var}(L_{\mathrm{R}})} = \frac{1}{k} \tag{4}$$

EC's benefit. The load variance of erasure coding is one-kth that of replication, which proves the potential of load balance of erasure coding in LLM inference.

4 Design

Main idea. Analysis in §3.2 motivates us to propose ECPrefix , which is an erasure-coded prefix KV cache mechanism tailored to distributed LLM inference scenarios, with the goal of resolving performance degradation from straggler nodes.

In this section, ECPrefix has the following design goals:

- **Facilitating Erasure Coding for Prefix Cache.** ECPrefix optimizes object size threshold via profiling and organizes prefix KV cache into the corresponding object, splits the object into data/parity blocks, and distributes them across nodes (§4.1).
- **Reducing Latency by Adaptive Stripe Size.** ECPrefix uses a dynamic programming model to adaptively optimize erasure coding parameters k (data blocks) and m (parity blocks) based on object size and access hotness (§4.2).
- **Accelerating Erasure-Coded Reading with Pipelining.** ECPrefix leverages the linearity of erasure decoding to pipeline transmission and partial decoding, overlapping network I/O with CPU computation to mitigate decoding overhead (§4.3).

4.1 Facilitating Erasure Coding for Prefix Cache

Architecture. Figure 3 shows the architecture of ECPrefix . ECPrefix has two processes: Write and Read. In the write process, ECPrefix profiles prefix KV cache and organizes each prefix KV cache into the corresponding object and adaptively selects the stripe size parameter of k and m (see details in §4.2). At last, ECPrefix encodes k data blocks for m parity blocks and distributes the total $(k+m)$ blocks evenly across nodes. In the read process, ECPrefix leverages erasure code's arbitrary reconstruction property to avoid straggler nodes and retrieve the complete prefix KV cache, accelerated by pipelining reading (see details in §4.3).

Organizing prefix KV caches into objects. As stated in §2.2, ECPrefix first needs to organize prefix KV caches into objects. A straightforward way is to treat each prefix KV cache as a single object, but for those large prefix KV cache, we must split them into multiple small objects. The reason is that: (i) if we consider the large cache as one single object, then reading part of the cache must load and decode the whole object (see §2.2), thus amplifying the read and increasing read latency; (ii) when reuse requests target nodes hosting such large single objects, these nodes incur sustained high bandwidth consumption which risks turning them into stragglers. Such node-level straggling creates discrepancies in processing latency across the cluster, disrupting load balance and ultimately increasing tail latency. Here, we only need to determine the maximum threshold for object size without worrying about the metadata management of excessive objects, as each object's metadata (<1 KB [8]) is negligible compared to the prefix KV cache itself (>50 MB in usual [24]) and management costs remain low due to its ephemeral lifespan [24].

Determining Object Size via Profiling. To accurately determine the maximum object size threshold and address the aforementioned bottlenecks for prefix KV cache in LLM serving, we propose a two-phase profiling scheme: "initial simple profiling + runtime dynamic stratified optimization". Without historical access data at startup, we cannot identify hotspot caches and first rely on simple profiling to quickly obtain a basic size distribution; after accumulating access records, we iteratively optimize via stratified profiling. Hotspot prefix caches are the core cause of load imbalance [17]. The scheme proceeds as follows:

Step ①: Initial simple profiling. At startup, the size of all stored prefix caches is recorded, from which we can obtain the complete full-size distribution of cache. Based on this distribution, we set the 90th percentile of the distribution for maximum object size threshold S_{max}, ensuring the large cache is organized into multiple small objects.

Step ②: Runtime dynamic stratified optimization. As inference tasks proceed, access frequencies of each cache are accumulated to filter out the hot prefix cache (top 15% by access rate, accounting for more than 80% of total reuse requests [24]). Thereafter, instead of relying on full-size distribution, the size distribution of hotspot caches is used as a replacement to redefine and dynamically update the maximum object size threshold S_{max}.

m \ k	3	4	5	6	7	8
1	1.116	1.084	1.053	1.078	1.105	1.151
2	1.128	1.090	1.056	1.029	**1.000**	1.026
3	1.260	1.209	1.152	1.095	1.083	1.107

Table 1: Normalized latency with different stripe size k and m.

4.2 Reducing Latency by Adaptive Stripe Size

Insight. In erasure-coded prefix KV cache systems, the stripe size (i.e., the number of data blocks k and the number of parity blocks m) is a critical factor influencing load balancing performance and robustness against node congestion. To verify the insight, we conduct another measurement analysis on a cluster of 16 nodes with the same setting as that in §5. Table 1 shows the results of the normalized latency (TTFT), which evaluates the performance under different stripe sizes.

We can see that when k is too large or too small, the latency increases, as a small k may incur load imbalance and amplify congestion impacts, while a large k leads to worse tail latency. More importantly, we need to tune m under the same k for optimized latency and congestion resilience. For example, when $k = 4$ or 5, the normalized latency is minimal when $m = 1$; when $k = 6$ or 7, the normalized latency is minimal when $m = 2$. The reason is

that a low m fails to mitigate straggler nodes and cannot tolerate node congestion, while a high m incurs excessive storage overhead. Therefore, we need to tune both k and m to minimize ECPrefix 's latency while ensuring resilience against node congestion. For example, the normalized latency in Table 1 reaches its minimum when $k = 7$ and $m = 2$ balances load distribution, decoding complexity, and congestion tolerance.

To minimize single-request latency in erasure-coded prefix caches with congestion resilience, we propose a dynamic programming (DP) model that adaptively optimizes erasure coding parameters k (data blocks) and m (parity blocks), incorporating KV cache attributes, system constraints, and node congestion characteristics. State variables (inputs) include S (object size, $S \leq S_{max}$) and h (access hotness, dimensionless, $h \in [1, H_{\max}]$); system intrinsic parameters are B (network bandwidth), δ (decoding coefficient), N (max cluster nodes), and ω (storage overhead threshold, $\frac{k+m}{k} \leq \omega$); decision variables are k (positive integer, $1 \leq k \leq N$) and m (positive integer, $1 \leq m \leq \lfloor(\omega - 1)k\rfloor$).

Transmission time should depend on the fastest k blocks among the $k + m$ blocks. Assuming block transmission times follow independent exponential distributions with rate $\omega = \frac{k \cdot B}{S}$, the expected transmission time is derived from order statistics.

For n exponential random variables, the expected k-th order statistic is:

$$\mathbb{E}[T_{(k)}] = \frac{1}{\omega}(H_n - H_{n-k}) \tag{5}$$

where H_m is the m-th harmonic number.

To obtain a closed-form expression, we employ the well-known approximation of harmonic numbers [20]. For m, the harmonic number H_m can be approximated as:

$$H_m \approx \ln m + \gamma \tag{6}$$

where γ is the Euler-Mascheroni constant. Then we can conclude the following expression:

$$\mathbb{E}[T_{(k)}] \approx \frac{1}{\omega}[\ln(k + m) - \ln(m)] \tag{7}$$

$$= \frac{1}{\omega} \ln\left(1 + \frac{k}{m}\right) \tag{8}$$

Incorporating system parameters $\frac{1}{\omega} = \frac{S}{k \cdot B}$ and access hotness h:

$$T_{\text{transmission}} = h \cdot \frac{S}{k \cdot B} \cdot \ln\left(1 + \frac{k}{m}\right) \tag{9}$$

The $\ln(1 + k/m)$ term quantifies how erasure coding achieves load balance and reduces waiting time: larger m enables faster completion by skipping slow nodes. Decoding time depends on k.

$$T_{\text{Decoding}} = \delta \cdot k \tag{10}$$

Total latency $L(S, h, k, m)$ is the sum of transmission time and decoding time:

$$L(S, h, k, m) = h \cdot \frac{S}{k \cdot B} \cdot \ln\left(1 + \frac{k}{m}\right) + \delta k \tag{11}$$

For DP optimization, we discretize S into $\{S_1, ..., S_n\}$ ($S_i = i \cdot \Delta S$) and h into $\{h_1, ..., h_p\}$ (hotness levels), forming state set $\{(S_i, h_j)\}$. Define $L^*(S_i, h_j)$ as the minimum latency for state (S_i, h_j), with optimal substructure:

$$L^*(S_i, h_j) = \min_{\substack{1 \leq k \leq K_{\max} \\ 1 \leq m \leq \lfloor(\omega-1)k\rfloor}} L(S_i, h_j, k, m) \tag{12}$$

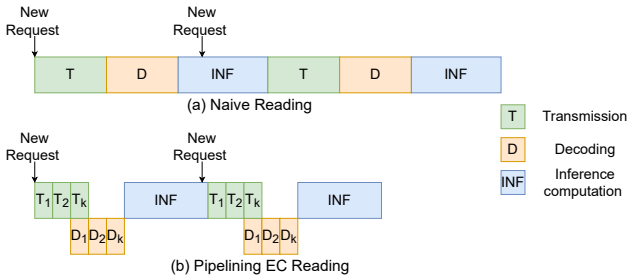


Figure 4: Pipelining erasure-coded reading can reduce the latency.

We denote by (k^*, m^*) the optimal erasure coding configuration for state (S_i, h_j):

$$(k^*, m^*) = \arg \min_{\substack{1 \leq k \leq N-m \\ 1 \leq m \leq \lfloor(\omega-1)k\rfloor}} \left(\frac{h \cdot S}{k \cdot B} \cdot \ln\left(1 + \frac{k}{m}\right) + \delta k\right). \tag{13}$$

Since k and m are discrete parameters with limited ranges, our DP framework evaluates all feasible (k, m) pairs and selects the configuration achieving the minimum latency.

4.3 Pipelining Erasure-Coded Reading

Limitation of EC. Despite achieving better load balance, EC increases inference latency due to additional computation required for decoding. As shown in Figure 4(a), a naive EC prefix cache method, when retrieving cache blocks, cannot directly obtain the complete object if the collected blocks include parity blocks, it must rely on decoding operations until exactly k blocks are collected. The time overhead introduced by this decoding operation may significantly increase the latency of the entire inference system. It is noted that the writing of the Prefix KV cache does not affect TTFT. Consequently, our focus is solely on decreasing load latency.

Pipelining EC read via its linearity. To mitigate this, we leverage the fact that erasure decoding is a linear algebraic process that supports partial computation, enabling the overlap of transmission and decoding. As shown in Figure 4(b), this property enables a pipelining erasure-coded reading mechanism, where T_k represents the arrival time of k-th block and D_k represents the decoding of k-th block. The network bandwidth and CPU computing resources are utilized synergistically with the following principles:

- Intermediate incremental refinement: The system initiates partial decoding computations D_1 as soon as the first two blocks arrive at T_2, progressively refining an intermediate state with each new block, rather than waiting for k blocks to accumulate, which synchronizes decoding progress with block transmission and eliminates the idle time between data arrival and computation initiation.
- Completion via arbitrary reconstruction: As long as any k blocks have been transmitted on demand at T_k, ECPrefix tries to reconstruct the complete object without attempting to receive redundant blocks according to the arbitrary reconstruction property of erasure coding, which reduces the occurrence of read amplification and ensures fast decoding.
- Zero-overhead for GPU: All decoding computations $(D_1, D_2, .., D_k)$ are isolated to CPU cores, ensuring no contention with GPU's inference tasks. This is because the GPU resource is primarily significant in LLM serving.

5 Evaluation

In this section, we evaluate the efficiency performance of ECPrefix to answer the following questions:

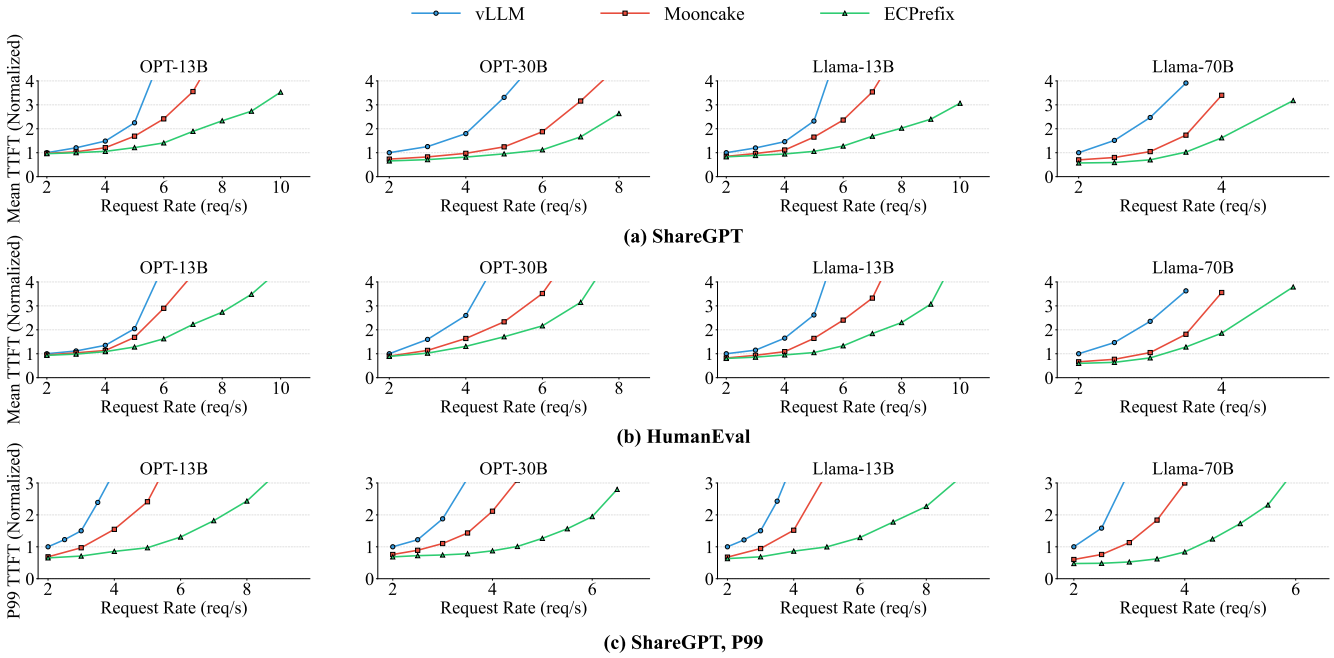


Figure 5: (Experiment 1) Latency performance upon ShareGPT and HumanEval.

- How much does ECPrefix reduce the latency compared to other methods? (Exp 1)
- How much does ECPrefix improve the load imbalance? (Exp 2)
- Does the proposed design improve ECPrefix 's performance? (Exp 3)
- How does ECPrefix perform with long context input? (Exp 4)
- Can ECPrefix maintain high efficiency under the scenario of more nodes? (Exp 5)

5.1 Experimental Setup

Environment: We conduct experiments on machines equipped with 2 NVIDIA GPUs A100(80GB), 36-core Intel Xeon Platinum 8352V, 512 GB DRAM, 2x1.5TB SSDs with PCIe Gen 4 interfaces. Machines are interconnected via a 25 Gbps Mellanox ConnectX-5 InfiniBand network. We use ISA-L [10], vLLM v0.8.0 [23] , LMCache v0.2.0 [4], Pytorch v2.7 [16] and CUDA v12.8 [13].

Workload: We use two representative open-source models in our experiments, including Llama-2 [22] and OPT [27] with different sizes. All the models are FP-16 formatted versions. We served the 13B models with a single GPU, and the model with 30B parameters and 70B parameters are evaluated using tensor parallelism. We conduct experiments of our work upon ShareGPT dataset[21] and HumanEval [2] in comparison to vllm and state-of-the-art method (Replication [17]).

5.2 Performance result

Experiment 1 (Inference latency). We evaluate the latency (Mean TTFT and P99 TTFT) and request rate with OPT models and Llama-2 models under increasing client requests. Following established methodology [26], we report the latency by the throughput. Note that all results are normalized compared to vLLM's TTFT with 2 requests per second. Figure 5 presents the normalized TTFT (y-axis) and request rate (x-axis) for two different datasets. Overall, ECPrefix can improve the latency (TTFT) in the four models under the same throughput compared to replication and vLLM. However, the benefit from ECPrefix is negligible when the request rate is low, since the probability of load imbalance occurring is also low in this scenario. As the request rate increases, ECPrefix demonstrates a better latency trending, which claims the effectiveness of our techniques.

Specifically, on ShareGPT, Figure 5(a) shows that ECPrefix reduces the mean TTFT by 46.8% in OPT-13B and 52.3% in Llama-70B compared to replication, when the request rate equals 7 and 4 req/s. On HumanEval, Figure 5(b) shows that ECPrefix reduces the mean TTFT by 43.9% in OPT-13B and 47.6% in Llama-70B compared to replication when the request rate equals 6 and 4 req/s. Although latency rises for all methods, ECPrefix demonstrates superior effectiveness by maintaining the lowest growth rate. Furthermore, the result between different datasets show the improvement of ECPrefix in longer input dataset. This is because ECPrefix can cope with the various length contexts.

In addition, Figure 5(c) shows that ECPrefix can significantly reduce the tail latency. For Llama-13B, ECPrefix can reduce the P99 TTFT by 70.9% compared to replication when the request rate equals 4 req/s. For the largest model Llama-70B, ECPrefix can reduce the P99 TTFT by up to 73% compared to replication when the request rate equals 4 req/s. As the request rate increases, the tail latency of all three methods escalates, with ECPrefix exhibiting the most marginal increase. More importantly, the reduction of tail latency is higher than the mean latency. This result ensures our motivation for tolerating straggler nodes.

Experiment 2 (Load imbalance). We evaluate the distribution of loads across nodes by using the load imbalance metric λ [18].

$$\lambda = \frac{L_{max} - L_{avg}}{L_{avg}} \times 100\% \tag{14}$$

where L_{max} is the load on the node which is maximally loaded and L_{avg} is the load on any node under a perfect scheme, where the total load is equally distributed among all the servers without any overhead. λ measures the percentage of additional load on

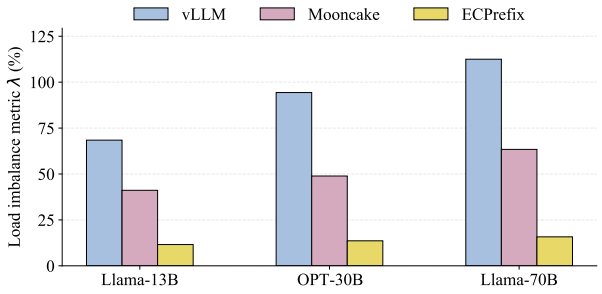


Figure 6: (Experiment 2) Load imbalance in ShareGPT.

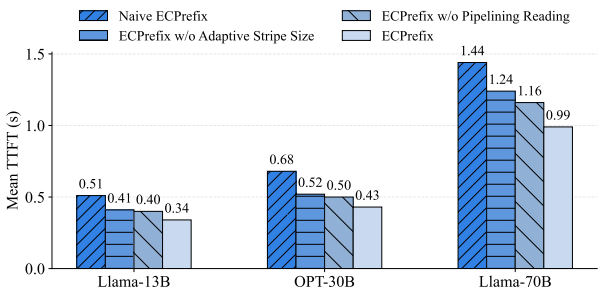


Figure 7: (Experiment 3) Ablation study.

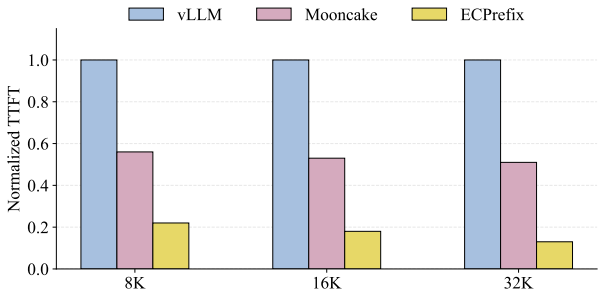


Figure 8: (Experiment 4) Latency with long context length.

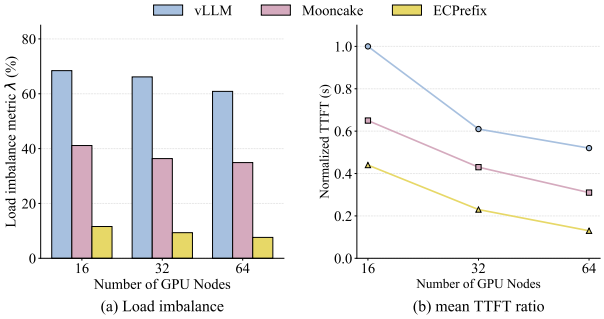


Figure 9: (Experiment 5) Performance on more nodes.

the maximally loaded node as compared to the ideal average load. Lower values of λ are better.

Figure 6 shows that the λ of ECPrefix maintains the minimum among all compared methods, demonstrating its superior load balancing capability. For Llama-13B, compared with Replication and vLLM, ECPrefix 's load imbalance metric reduces by 71.8% and 83.1%, respectively. For the larger OPT-30B model, ECPrefix 's metric is 72.2% lower than replication and 81.7% lower than vLLM. For Llama-70B model, ECPrefix still maintains the optimal performance: its metric is 15.8%. When the model scale increases, the load imbalance metric of all three methods shows a slight upward trend, but ECPrefix maintains the smallest growth amplitude. This confirms the effectiveness of ECPrefix 's load balancing design, as it can mitigate the load concentration caused by model scale expansion and node congestion.

Experiment 3 (Ablation study). To evaluate the effectiveness of design proposed in §4.2 and §4.3, we conduct evaluations on different models with various ECPrefix , including Naive ECPrefix (without adaptive stripe size and pipelining reading), ECPrefix without adaptive stripe size, ECPrefix without pipelining reading, ECPrefix . Through these evaluations, we are able to separately evaluate the performance gain by each scheme. Note that the profiling in §4.1 is fundamental to §4.2, which will be evaluated together.

Figure 7 shows the result: Compared to ECPrefix without adaptive stripe size (§4.2), ECPrefix decreases the mean TTFT by 17.6%-20.2% on Llama-13B, OPT-30B, Llama-70B; Compared to ECPrefix without pipelining reading mechanism, ECPrefix decreases in latency d of 16.5%-16.9% for Llama-13B, OPT-30B, and Llama-70B. The result validates the effectiveness of our design, so ECPrefix can use the proposed design to further enhance performance.

Experiment 4 (Long Context). We evaluate the latency with maximum context length from 8k, 16k to 32k with scaling ShareGPT, while the minimum context length has also increased accordingly. Note that all results are normalized compared to vLLM's TTFT.

As shown in Figure 8, compared to replication, ECPrefix decreases the TTFT by 60.7% for the maximum length of 8k, and 74.1% for the maximum length of 32k. As the context length increases, the gap between ECPrefix and replication becomes more pronounced, which demonstrates ECPrefix 's effectiveness in the long context.

Experiment 5 (Scalability). We evaluate the load imbalance and latency by varying the number of nodes used in the inference on Llama-13B for ShareGPT dataset. We conduct experiments with 16, 32, and 64 GPUs, respectively, and measure load imbalance and the mean TTFT ratio of vllm, replication (Mooncake), and ECPrefix . The newly added node will be employed by data parallelism.

Figure 9(a) shows that, as the number of nodes increases, load imbalance in all methods decreases; ECPrefix still maintains the lowest load imbalance. Furthermore, ECPrefix achieves a greater reduction in load imbalance compared to the replication approach from 16 nodes to 64 nodes. Since more nodes enable the generation of larger stripes, leading to a smaller theoretical variance of the encoding. In addition, Figure 9(b) shows that ECPrefix maintains the lowest TTFT with a high effective prefix KV cache system, which shows ECPrefix 's potential for large language model serving with massive nodes.

6 Conclusion

This paper presents ECPrefix : a new prefix KV cache framework based on erasure coding (instead of replication), which distributes encoded blocks of hot prefix caches (organized as profile-guided objects) across nodes, along with adaptive striping and pipelined reading optimizations. Evaluation shows that ECPrefix reduces TTFT by up to 52.3% over existing systems. In the future, we will explore the potential of ECPrefix in terms of Fine-grained storage, such as a hybrid load balancing strategy for prefix KV cache under different storage media, such as GPU HBM, CPU DRAM, and SSD.

Acknowledgments

The corresponding author is Yuchong Hu. This work was supported by the National Natural Science Foundation of China (No. 62272185, No. U25A20423), and the Key Laboratory of Information Storage System, Ministry of Education of China.

References

[1] Moitra A. Super-resolution, extremal functions and the condition number of vandermonde matrices. *Proceedings of the forty-seventh annual ACM symposium on Theory of computing*, pages 821–830, 2015.

[2] Mark Chen. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[3] Weijian Chen, Shuibing He, Haoyang Qu, Ruidong Zhang, Siling Yang, Ping Chen, Yi Zheng, Baoxing Huai, and Gang Chen. {IMPRESS}: An {Importance-Informed}{Multi-Tier} prefix {KV} storage system for large language model inference. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pages 187–201, 2025.

[4] Yihua Cheng, Yuhan Liu, Jiayi Yao, Yuwei An, Xiaokun Chen, Shaoting Feng, Yuyang Huang, Samuel Shen, Kuntai Du, and Junchen Jiang. Lmcache: An efficient kv cache layer for enterprise-scale llm inference. *arXiv preprint arXiv:2510.09665*, 2025.

[5] Kuntai Du, Bowen Wang, Chen Zhang, Yiming Cheng, Qing Lan, Hejian Sang, Yihua Cheng, Jiayi Yao, Xiaoxuan Liu, Yifan Qiao, et al. Prefillonly: An inference engine for prefill-only workloads in large language model applications. *arXiv preprint arXiv:2505.07203*, 2025.

[6] Sarah Fakhoury, Aaditya Naik, Georgios Sakkas, Saikat Chakraborty, and Shuvendu K Lahiri. Llm-based test-driven interactive code generation: User study and empirical evaluation. *IEEE Transactions on Software Engineering*, 2024.

[7] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. Prompt cache: Modular attention reuse for low-latency inference. *Proceedings of Machine Learning and Systems*, 6:325–338, 2024.

[8] Yuchong Hu, Liangfeng Cheng, Qiaori Yao, Patrick PC Lee, Weichun Wang, and Wei Chen. Exploiting combined locality for wide-stripe erasure coding in distributed storage. In *19th USENIX Conference on File and Storage Technologies*, pages 233–248, 2021.

[9] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. Deepspeed ulysses: System optimizations for enabling training of extreme long sequence transformer models. *arXiv preprint arXiv:2309.14509*, 2023.

[10] Intel Storage Acceleration Library. https://github.com/intel/isa-l.

[11] Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.

[12] Subramanian Muralidhar, Wyatt Lloyd, Sabyasachi Roy, Cory Hill, Ernest Lin, Weiwen Liu, Satadru Pan, Shiva Shankar, Viswanath Sivakumar, Linpeng Tang, et al. f4: Facebook's warm blob storage system. In *11th USENIX Symposium on Operating Systems Design and Implementation*, pages 383–398, 2014.

[13] NVIDIA. https://developer.nvidia.com/cuda-toolkit.

[14] OpenAI. Chatgpt. https://openai.com/index/chatgpt/, Feb 2024.

[15] Zuo P, Lin H, Deng J, and et al. Serving large language models on huawei cloudmatrix384. *arXiv preprint arXiv:2506.12708*, 2025.

[16] Pytorch. https://github.com/pytorch/pytorch.

[17] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Feng Ren, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. Mooncake: Trading more storage for less computation—a {KVCache-centric} architecture for serving {LLM} chatbot. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*, pages 155–170, 2025.

[18] KV Rashmi, Mosharaf Chowdhury, Jack Kosaian, Ion Stoica, and Kannan Ramchandran. {EC-Cache}:{Load-Balanced},{Low-Latency} cluster caching with online erasure coding. In *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*, pages 401–417, 2016.

[19] Zhirong Shen, Yuhui Cai, Keyun Cheng, Patrick P. C. Lee, Xiaolu Li, Yuchong Hu, and Jiwu Shu. A survey of the past, present, and future of erasure coding for storage systems. *ACM Trans. Storage*, 21(1):4:1–4:39, 2025.

[20] Zhi-Wei Sun. Arithmetic theory of harmonic numbers. *Proceedings of the American Mathematical Society*, 140(2):415–428, 2012.

[21] ShareGPT teams. https://sharegpt.com.

[22] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[23] vllm. https://github.com/vllm-project/vllm.

[24] Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyuan Yu, and Haibo Chen. Kvcache cache in the wild: Characterizing and optimizing kvcache cache at a large cloud provider. *arXiv preprint arXiv:2506.02634*, 2025.

[25] Stephen B Wicker and Vijay K Bhargava. *Reed-Solomon codes and their applications*. John Wiley & Sons, 1999.

[26] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538, 2022.

[27] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, et al. Opt: Open pre-trained transformer language models. *arXiv preprint arXiv:2205.01068*, 2022.

[28] Yang Zhang, Hanlei Jin, Dan Meng, Jun Wang, and Jinghua Tan. A comprehensive survey on process-oriented automatic text summarization with exploration of llm-based methods. *arXiv preprint arXiv:2403.02901*, 2024.